



class

Programmer:

Beatriz Orozco G.

Herencia

y tipos abstractos

en Python

Programación II · Sesión de 2 horas

Facultad de Ingeniería · UAEMéx

Agenda de la sesión

Dos horas · seis bloques · teoría, ejemplos y práctica guiada



1. Repaso de POO y motivación

10 min



2. Herencia simple en Python

20 min



3. super(), MRO y herencia múltiple

20 min



4. Polimorfismo y duck typing

15 min



5. Tipos abstractos: ABC y abstractmethod

30 min



6. Práctica guiada y cierre

25 min

Objetivos de aprendizaje

Al final de la sesión, cada estudiante podrá:

- 1 Diseñar jerarquías de clases usando herencia simple y múltiple en Python.
- 2 Sobrescribir métodos y aprovechar `super()` para reutilizar comportamiento.
- 3 Distinguir polimorfismo de duck typing y aplicar ambos en código real.
- 4 Construir clases base abstractas con el módulo `abc` para definir contratos.
- 5 Reconocer cuándo conviene usar herencia y cuándo prefiere otra alternativa.

Repaso rápido: los cuatro pilares

Ya conocemos los tres primeros · hoy profundizamos en herencia y abrimos polimorfismo



Abstracción

Modelar lo esencial de un problema con clases.



Encapsulación

Ocultar el estado interno; exponer interfaz.



Herencia

Crear clases nuevas a partir de otras existentes.



Polimorfismo

Una misma interfaz, comportamientos distintos.

*Hoy nos enfocamos en **Herencia** + **Polimorfismo** y los habilitamos con tipos abstractos.*

¿Qué es la herencia?

Una clase hija obtiene atributos y métodos de una clase padre, y puede extenderlos o modificarlos

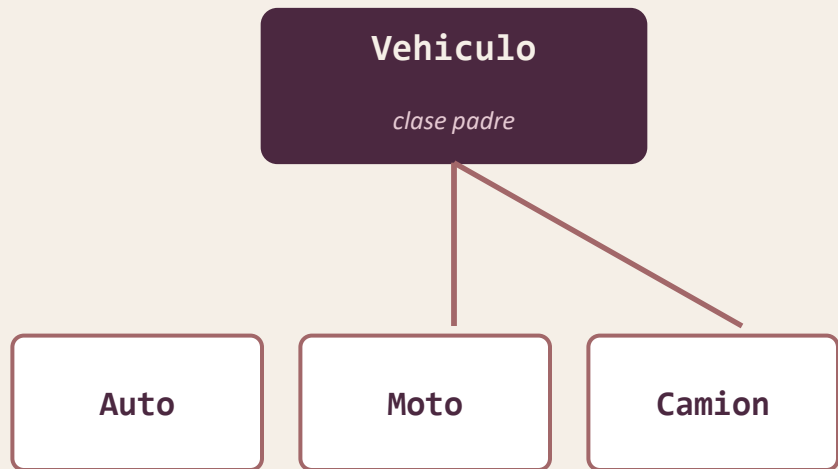


Idea central

Si dos clases comparten estructura y comportamiento, podemos extraer lo común a una **clase padre** y dejar que las **clases hijas** hereden esa base.

Beneficios:

- Reutilización de código
- Modelo cercano a la realidad
- Especialización progresiva
- Mantenimiento centralizado



hijas — heredan de Vehiculo y agregan lo propio

Sintaxis: herencia simple

class Hija(Padre): · todo lo público y protegido del padre queda disponible



```
class Vehiculo:
    def __init__(self, marca, velocidad=0):
        self.marca = marca
        self.velocidad = velocidad
    def acelerar(self, delta):
        self.velocidad += delta

class Auto(Vehiculo):
    def abrir_cajuela(self):
        print(f"Cajuela del {self.marca} abierta")
```

Lo que hereda Auto



Atributo marca y velocidad



Método acelerar(delta)



Constructor __init__



Atributos privados con __

auto = Auto("Toyota")

Sobrescritura y super()

La hija puede redefinir un método y, si quiere, llamar a la versión del padre



```
class Auto(Vehiculo):
    def __init__(self, marca, puertas=4):
        super().__init__(marca)
        self.puertas = puertas

    def acelerar(self, delta):
        # limita velocidad antes de delegar
        delta = min(delta, 20)
        super().acelerar(delta)
```

`super().__init__(...)`

Llama al constructor del padre. Reutiliza la inicialización en lugar de copiarla.

Sobrescritura (override)

Auto.acelerar reemplaza a Vehiculo.acelerar, pero conserva la base con super().

Tipos de herencia

Python soporta los cuatro · cada uno con un caso de uso típico

Simple

Una hija, un padre.

Auto → Vehiculo

Multinivel

Cadena de herencia.

Deportivo → Auto → Vehiculo

Jerárquica

Varias hijas, un padre.

Auto, Moto, Camion → Vehiculo

Múltiple

Una hija, varios padres.

AnfibioAuto → Auto, Bote

Herencia múltiple en Python

Útil con clases pequeñas y enfocadas (mixins) · evita ambigüedades con MRO



```
class Volador:
    def volar(self): print("vuela")

class Nadador:
    def nadar(self): print("nada")

class Pato(Volador, Nadador):
    pass

p = Pato()
p.volar(); p.nadar()  # vuela / nada
```

Recomendaciones



Usa mixins: clases pequeñas con un solo propósito.



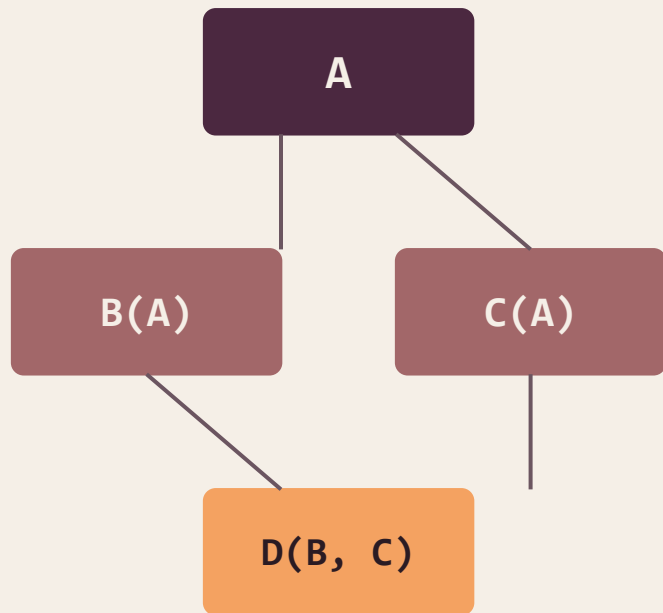
Evita firmas distintas para el mismo método en padres.



Si dudas, prefiere composición sobre herencia múltiple.

MRO: Method Resolution Order

Cuando un método existe en varios padres, Python usa el algoritmo C3 para decidir



```
D.__mro__  
# (D, B, C, A, object)
```

```
D.mro()  
# útil para depurar
```

Reglas del MRO

- Las hijas aparecen antes que sus padres.
- Se respeta el orden de declaración de los padres.
- object queda siempre al final de la cadena.

Polimorfismo en acción

El mismo mensaje produce respuestas distintas según el tipo del objeto



```
class Animal:
    def hablar(self): ...

class Perro(Animal):
    def hablar(self): return "Guau"

class Gato(Animal):
    def hablar(self): return "Miau"

for a in [Perro(), Gato()]:
    print(a.hablar())
```



Idea

El bucle no necesita saber si el objeto es Perro o Gato. Solo le pide hablar().

Salida:



```
Guau
Miau
```

Duck typing

"If it walks like a duck and quacks like a duck, it's a duck."

La idea

Python no exige una clase base común. Si un objeto tiene los métodos esperados, sirve.

Ventajas

- Código más flexible y conciso
- No exige jerarquías profundas
- Pero requiere disciplina y pruebas



```
def hacer_hablar(x):  
    print(x.hablar())  
  
class Robot:  
    def hablar(self): return "Bip"  
  
hacer_hablar(Perro()) # Guau  
hacer_hablar(Robot()) # Bip – sin  
herencia común
```

¿Para qué sirven los tipos abstractos?

Para definir contratos: qué métodos debe tener una familia de clases, sin importar cómo los implementen



Problema sin abstracción

- Olvidar implementar un método clave en una hija.
- Crear instancias de clases pensadas como base.
- Errores que solo se ven en tiempo de ejecución.
- Falta de claridad sobre qué se debe implementar.



Solución con ABC

- Marcas la clase base como abstracta: no se puede instanciar.
- Declaras qué métodos son obligatorios con `@abstractmethod`.
- Si una hija olvida un método, falla al crear el objeto, no después.
- Documenta el contrato de la jerarquía.

El módulo abc en Python

Importa ABC y abstractmethod desde la biblioteca estándar

```
from abc import ABC, abstractmethod

class FiguraGeometrica(ABC):

    @abstractmethod
    def area(self): ...

    @abstractmethod
    def perimetro(self): ...

    def describir(self): # método concreto compartido
        print(f"Área={self.area()}, P={self.perimetro()}")
```

Tres elementos clave

ABC

Clase base que vuelve abstracta a la nuestra.

@abstractmethod

Marca métodos obligatorios para las hijas.

Métodos concretos

Pueden coexistir y ser reutilizados.

Implementando una clase abstracta

Las hijas deben implementar todos los métodos abstractos para poder instanciarse



```
class Circulo(FiguraGeometrica):
    def __init__(self, r): self.r = r
    def area(self): return 3.1416 * self.r ** 2
    def perimetro(self): return 2 * 3.1416 * self.r

class Rectangulo(FiguraGeometrica):
    def __init__(self, b, h): self.b, self.h = b, h
    def area(self): return self.b * self.h
    def perimetro(self): return 2 * (self.b + self.h)
```



Uso



```
c = Circulo(3)
r = Rectangulo(4, 5)

for f in [c, r]:
    f.describir()

# Área=28.27, P=18.85
# Área=20, P=18
```

El error que ABC detecta a tiempo

Sin abstractmethod, este bug aparecería tarde · con ABC, falla al instanciar



```
class Triangulo(FiguraGeometrica):  
    def __init__(self, b, h):  
        self.b, self.h = b, h  
    def area(self): return self.b * self.h / 2  
    # falta perimetro()
```

```
t = Triangulo(3, 4)  
# TypeError: Can't instantiate abstract class  
#   Triangulo with abstract method perimetro
```



Garantías



El error aparece al construir el objeto, no al usarlo.



Se puede crear FiguraGeometrica como contrato y nada más.



Más fácil documentar y revisar la jerarquía.

Propiedades y atributos abstractos

Se pueden combinar `@abstractmethod` con `@property` para exigir atributos calculados

```
class Empleado(ABC):
    def __init__(self, nombre):
        self.nombre = nombre

    @property
    @abstractmethod
    def salario(self): ...

class PorHora(Empleado):
    def __init__(self, nombre, h, t):
        super().__init__(nombre)
        self.h, self.t = h, t

    @property
    def salario(self): return self.h * self.t
```

Combinación clave

`@property`
`@abstractmethod`

Obliga a implementar un atributo de solo lectura calculado, ideal para reglas de negocio.

Acceso:

`emp.salario` (sin paréntesis)

ABC, Protocol y duck typing

Tres formas de hablar de “contratos” en Python · cada una con su lugar y momento

Duck typing

informal · muy flexible

- No declaras nada explícito.
- Confías en que el objeto tiene el método.
- Útil en código pequeño y dinámico.

ABC (abc.ABC)

contrato verificado al instanciar

- Herencia explícita y obligatoria.
- Falla pronto si falta un método.
- Ideal para jerarquías de dominio.

Protocol (typing)

tipado estructural · sin herencia

- Define forma esperada del objeto.
- Compatibilidad por estructura, no por herencia.
- Verifica con `mypy/Pyright`.

Buenas prácticas con herencia

Reglas heurísticas que evitan diseños frágiles



Hereda solo si “es un”

Auto es un Vehículo: sí. Lista es un Reporte: no.



Prefiere composición

Si la relación es “tiene un”, usa atributos en lugar de heredar.



Mantén jerarquías cortas

Más de 3-4 niveles suele ser señal de mal diseño.



Cuidado con herencia múltiple

Úsala con mixins pequeños y bien definidos.



No abusos de las ABC

Solo cuando varias clases compartirán el contrato.



Cuidado con `__init__` del padre

Evita que dependa de la hija; usa hooks o métodos plantilla.

Práctica guiada: sistema de pagos

25 minutos · trabajamos en parejas · entrega al final de la sesión



Tarea

Diseña una clase abstracta **MetodoPago** con métodos abstractos **validar()** y **cobrar(monto)**.

Implementa al menos:

- TarjetaCredito (valida 16 dígitos y CVV)
- Transferencia (valida CLABE de 18 dígitos)
- Efectivo (siempre válido, suma a una caja)



Criterios de evaluación

30%

Uso correcto de ABC y abstractmethod

25%

Implementación completa de las tres clases

20%

Función procesar_pago(metodo, monto) polimórfica

15%

Validaciones reales (no triviales)

10%

Pruebas en main que demuestren el flujo

Lo que nos llevamos

Cinco ideas para recordar y aplicar en cada proyecto

1

Herencia es para reutilizar y especializar

Cuando una clase es de verdad un caso particular de otra.

2

`super()` conecta a la hija con el padre

Sin duplicar lógica de inicialización ni de comportamiento.

3

Polimorfismo simplifica el código cliente

Un solo bucle, muchas implementaciones.

4

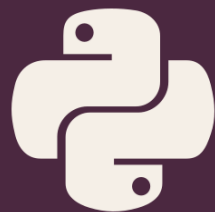
ABC convierte expectativas en contratos

Errores tempranos, jerarquías claras y autodocumentadas.

5

Composición a veces gana

No todo lo que se parece debe heredar; piensa antes de extender.



¿Preguntas?

Practiquen: hereden, sobrescriban, abstraigan.

M. en A.E.G. Beatriz Orozco Garduño

Coordinación de Docencia · Ingeniería en Computación · UAEMéx